



UFR MI Licence 2 MIAGE
Fondamentaux de la POO
Session1 (02h)
Année : 2023-2024



EXERCICE I : Champs et méthodes d'accès (07pts)

Recopiez le code et indiquez pour chacune des lignes celle comportant des erreurs et la raison et celle qui sont OK sur le modèle de a

a. Accessibilité à l'extérieur des packages //fichier A.java package p ; public class A { /*...*/ } Class B { /*...*/ } //fichier X package q ; public class X { p.A varDeTypeA ;//OK p.B varDeTypeA ;// <u>Erreur : B</u> <u>n'est pas visible hors de p</u> }	b. Classe héritable final class A { /*...*/ } class B extends A { /*...*/ } //.....
c. Modificateurs d'attributs class A { final int GRAV = 9,78 ;} A a = new A() ; int i = a.GRAV ;//.....OK..... a.GRAV++ ;//.....	d. Modificateur de méthode public class A { final void m() {} } class B extends A { void m() {} //..... }
e. Niveau d'accès package p ; public class A { public int a1 = 0 ; protected int a2 = 0 ; int a3 = 0 ; private int a4 = 0 ; } class A2 extends p.A { int x1 = this.a1 //..... int x2 = this.a2 //..... int x3 = this.a3 //..... int x4 = this.a4 //..... }	f. Niveau d'accès package p ; public class A { public int a1 = 0 ; protected int a2 = 0 ; int a3 = 0 ; private int a4 = 0 ; } class B { p.A a = new p.A() ; int x1 = a.a1 //..... int x2 = a.a2 //..... int x3 = a.a3 //..... int x4 = a.a4 //.....}

a4 est déclarée comme
privée dans la classe A
donc pas accessible depuis
la classe B

EXERCICE II : Etude de cas (13 pts)

Vous êtes chargé de développer un système pour gérer le personnel d'une entreprise. L'entreprise a plusieurs types d'employés, y compris des managers, des ingénieurs, et des employés de bureau. Chaque type d'employé a des attributs et des comportements différents.

Modéliser les employés :

- ✓ Créez une classe de base `Employe` avec des attributs communs tels que `id`, `nom`, `adresse`, et `salaire`.
- ✓ Définissez des méthodes dans `Employe` pour des actions communes comme `travailler()` et `augmenterSalaire()`.

Héritage et classes dérivées :

- ✓ Créez les classes dérivées : `Manager`, `Ingenieur`, et `EmployeDeBureau`, qui héritent de `Employe`.
- ✓ Chaque sous-classe devrait avoir des attributs et des méthodes supplémentaires qui sont pertinents pour son type. Par exemple, `Manager` pourrait avoir une liste d'employés sous sa responsabilité.

Généricité et gestion des équipes :

- ✓ Créez une classe `Equipe<T>` où `T` est un type d'`Employe`.
- ✓ Dans `Equipe<T>`, implémentez des méthodes pour ajouter des employés à l'équipe, obtenir le total des salaires, et décrire l'équipe.

Polymorphisme :

- ✓ Utilisez le polymorphisme pour permettre des comportements différents pour la méthode `travailler()` en fonction du type d'employé.
- ✓ Par exemple, un `Manager` pourrait coordonner des équipes, tandis qu'un `Ingénieur` pourrait résoudre des problèmes techniques.

1. Définissez la classe de base avec ses attributs et méthodes.
2. Implémentez `Manager`, `Ingenieur`, et `EmployeDeBureau` avec des attributs et comportements spécifiques.
3. Implémentez la générique pour gérer différents types d'employés au sein d'une équipe.
4. Ajoutez des méthodes pour gérer l'équipe.
5. Implémentez le comportement polymorphique de la méthode `travailler()`.
6. Créez une classe `Main` avec une méthode `main` pour des test.

Dr Konaté